Coding & Solution

Date	30 June 2026
Team ID	Not Provided
Project Name	Smart Lender
Maximum Marks	5 Marks

Coding & Solution

This document summarizes the implemented solution for Smart Lender, an ML-powered loan eligibility screening system. The solution combines data preprocessing, multi-model training, model selection, a Flask web interface, JSON APIs, batch CSV prediction, and automated tests. The repository demonstrates a working end-to-end pipeline from training to inference.

Instructions:

1. The codebase is structured into separate application, training, template, static, model, data, and test layers.
2. The implemented solution addresses the main problem statement: predicting loan approval based on applicant details and exposing the result through both web and API interfaces.
3. The repository includes all primary source files, dependency declarations, test files, model artifacts, templates, and setup instructions required to run the application locally.
4. A public repository URL is not available in the current workspace snapshot, so that field is documented transparently below.

Solution Summary

Field	Details
Repository Link / URL	https://github.com/YaswanthGanesh1234/SmartLender-SmartBridge.git

Programming Language(s)	Python, HTML, CSS, JavaScript
Framework(s) Used	Flask, scikit-learn, XGBoost, pandas, NumPy, matplotlib
Key Features Implemented	Loan approval prediction form, JSON prediction API, EMI calculator API, batch CSV upload and scoring, downloadable batch results, session-based history tracking, model comparison reporting, feature importance display, health endpoint, automated tests, and model training pipeline.
Pending / Incomplete Features	No database-backed persistence, no authentication or role-based access control, no deployment configuration, and no integrated formal load-testing tool configuration in the repository.
Setup / Run Instructions	Create and activate a virtual environment, install dependencies with `pip install -r requirements.txt`, train or refresh the model with `python train_model.py`, and run the app with `python app.py`. Open `http://127.0.0.1:5000` in a browser.

Code Quality Checklist

S.N	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	No

Additional Notes / Comments:

The application was executed successfully in the local environment and the health endpoint returned an `ok` status. The automated test suite also passed with `228 passed`, confirming that the current implementation is functionally stable.